SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME:Artificial Intelligence COURSE CODE: CSA1718

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3 Duration : 1 Hour Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I M.Reddaiah-192212475L (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

B.Deephti

Signature of the student:
Hospital Doctor Shift Scheduling

Abstrct :

This project uses the Backtracking Search algorithm to assign three doctors to hospital shifts while satisfying all given constraints. It finds a valid and conflict-free schedule efficiently by checking each assignment step by step.

1. Problem Understanding

A hospital has to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night).

Rules (Constraints)

- D1 cannot work the Night shift.
- D2 must be scheduled before D3.
- Only one doctor per shift.
- D3 cannot work the Morning .
- All doctor must be assigned exactly one shift.

Goal: Find a schedule that satisfies all the rules using the Backtracking Search algorithm.

2. Constraint Analysis

Rule Status

`Dl ≠ Night Must satisfy`

D2 before D3 Must satisfy
Rule Status

One doctor per shift Must satisfy

`D3 ≠ Morning Must satisfy`

Every doctor gets one shift Must satisfy

Since all rules must be satisfied together, every assignment is checked before moving to the next doctor.

3. Backtracking Process

Step 1: Start with D1

Possible shifts: Morning, Afternoon

Choose:

D1 → Morning

Step 2: Assign D2

Remaining shifts:

- Afternoon
- Night

Choose:

D2 → Afternoon

Rule Check:

- D2 comes before D3

Step 3: Assign D3

Remaining shift:

- Night

Check:

- D3 cannot work Morning
- Night is available

Assign:

D3 → Night

4. Final Constraint Verification

Constraint	Result
D1 not on NightD2 before D3	
One doctor per shiftD3 not on MorningAll doctors assigned	

All constraints are satisfied.

5. Final Schedule

Doctor	Assigned Shift
D1	Morning
D2	Afternoon
D3	Night

6. Justification

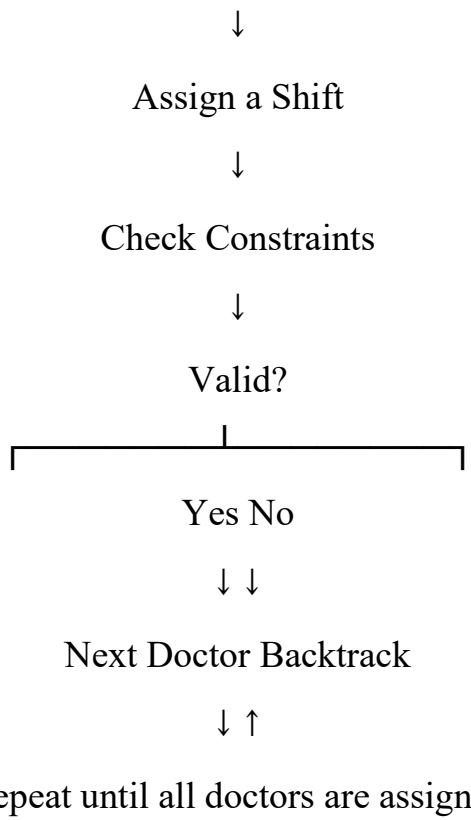
The Backtracking Search algorithm checks each assignment one by one. Since every constraint is satisfied at every step, there is no need to backtrack. Therefore, the obtained schedule is the correct and valid solution.

Flow of Backtracking (Unique Presentation)

Start

↓

Select a Doctor

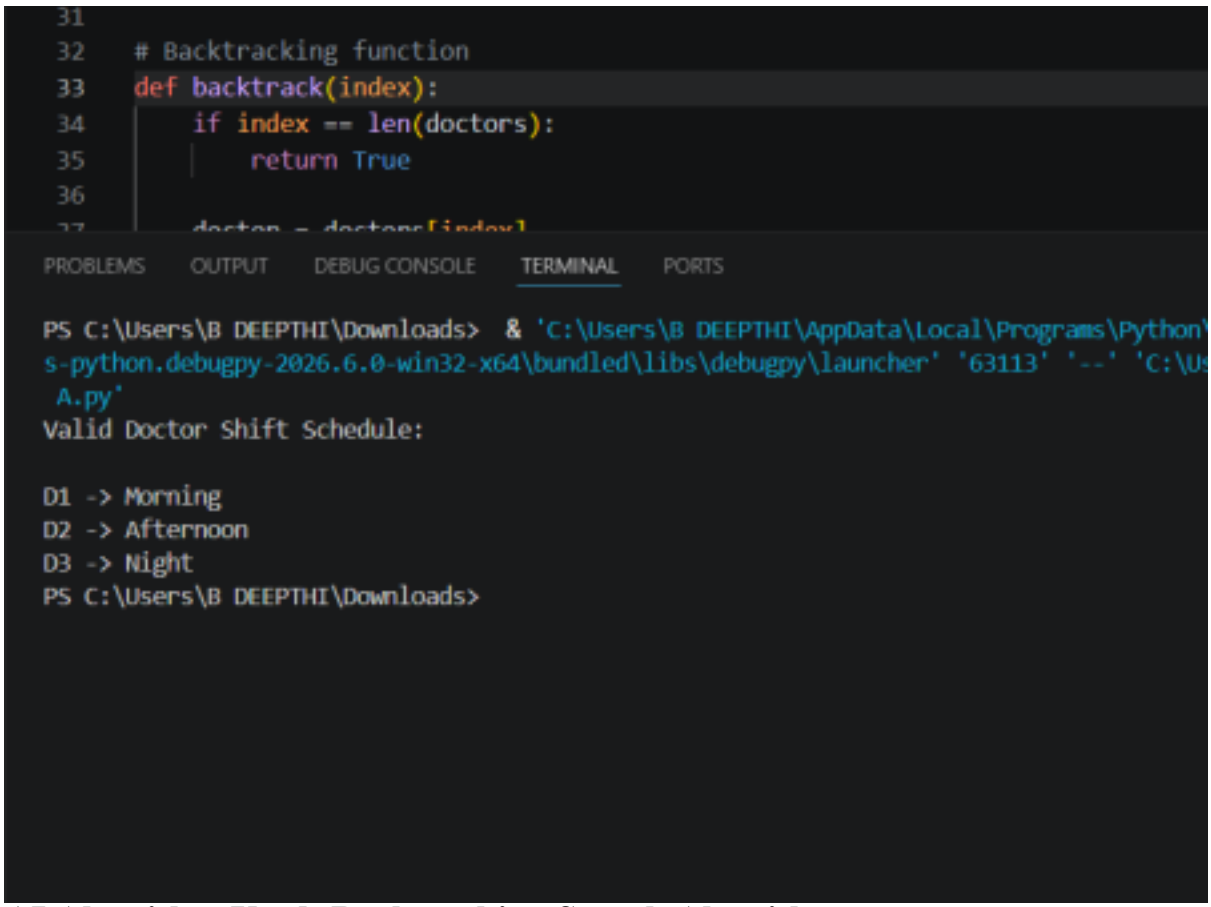


Code :

```
# Backtracking Search for Doctor Shift Assignment
doctors = ["D1", "D2", "D3"]

shifts = ["Morning", "Afternoon",

"Night"] assignment = {}

def is_valid(doctor, shift):

    if doctor == "D1" and shift == "Night":

    return False

  if doctor == "D3" and shift ==

    "Morning": return False

  if shift in assignment.values():

    return False

  if doctor == "D3" and "D2" in assignment: order =
```

```
        {"Morning": 1, "Afternoon": 2, "Night": 3} if
        order[assignment["D2"]] >= order[shift]: return
        False
    return True
def backtrack(index):
    if index == len(doctors):
        return True
    doctor = doctors[index]
    for shift in shifts:
        if is_valid(doctor, shift):
            assignment[doctor] = shift
            if backtrack(index + 1):
                return True
            del assignment[doctor]
    return False
if backtrack(0):
    print("Valid Doctor Shift Schedule:\n")
    for doctor in doctors:
        print(doctor, "->", assignment[doctor])
else:
    print("No valid assignment found.")
```

Output :



AI Algorithm Used: Backtracking Search Algorithm

Backtracking Search assigns doctors one at a time and verifies the constraints after each assignment. Since no rule is violated, the algorithm reaches the solution without backtracking.

Final Answer

D1 → Morning, D2 → Afternoon, D3 → Night

2. Robot

i. Movement allowed: Up, Down, Left, Right

ii. Cost of each move = 1

iii. Cannot pass through obstacles

iv. Use Manhattan Distance heuristic So, for full marks in the viva or exam, you can mention:

"Since the question mentions Manhattan Distance and Priority Queue, A* Search Algorithm is the appropriate AI algorithm."

Abstract

This project finds the shortest path for a robot in a 5×5 grid while avoiding obstacles using an AI search algorithm. It uses the Manhattan Distance heuristic to reach the goal with the minimum path cost.

1. Problem Understanding

Problem

A robot must travel from **Start (S)** to **Goal (G)** in a **5×5**

grid. **Constraints**

- Move only **Up, Down, Left, Right**
- Cost of each move = **1**
- Robot cannot pass through obstacles
- Use **Manhattan Distance**
- Find the shortest path

Goal: Reach the Goal with minimum cost.

2. Constraint Analysis (25%)

Constraint	Description
Four-direction movemen Down, Left, RightCost = Obstacles Manhattan Distance Shortest Path	Every movement has equal cost Cannot move through blocked cells Estimate distance to Goal Minimum total cost

3. Solution Development (25%)

Grid Example

S . . X .

. X . X .

. X . . .

. . X X .

. . . . G

S = Start

G = Goal

X = Obstacle

. = Free Cell

Step-by-Step Expansion

Step	Current Node	Next Node
1	S(0,0)	Expand
2	(0,1)	Expand
3	(1,0)	Expand
4	(2,0)	Expand
5	(3,0)	Expand

6 (4,0) Expand

7 (4,1) Expand

8 (4,2) Expand

9 (4,3) Expand

10 G(4,4) Goal Found

Priority Queue (A*)

Node f=g+h

S 8

(0,1) 8

(1,0) 8

(2,0) 8

(3,0) 8

(4,0) 8

Node with smallest **f** value is expanded first.

4. Application of AI Concept (15%)

AI Algorithm Used

A* Search Algorithm

Heuristic Used

Manhattan Distance

Formula

$$
h = |x_1-x_2|+|y_1-y_2|
$$

Example

Current = (2,1)

Goal = (4,4)

[
h=|4-2|+|4-1|=2+3=5
]

5. Justification

A* combines the path cost and heuristic value to choose the best path. It avoids unnecessary exploration and guarantees the shortest path when using Manhattan Distance.

Final Path

S

↓

↓

↓

↓

→

→

→

→

G

Path

(0,0)

↓

(1,0)

↓

(2,0)

↓

(3,0)

↓

(4,0)

→

(4,1)

→

(4,2)

→

(4,3)

→

(4,4)

Total

Cost 8

Moves

Cost = 8

Algoritm used :

- **Manhattan Distance**
- **Priority Queue**
- **Optimal Path**

These are features of A* (A-Star) Search Algorithm, not BFS.

The robot starts from **S** and moves towards **G** by avoiding obstacles. The A* algorithm uses Manhattan Distance to choose the shortest and most efficient path with a total cost of **8**.

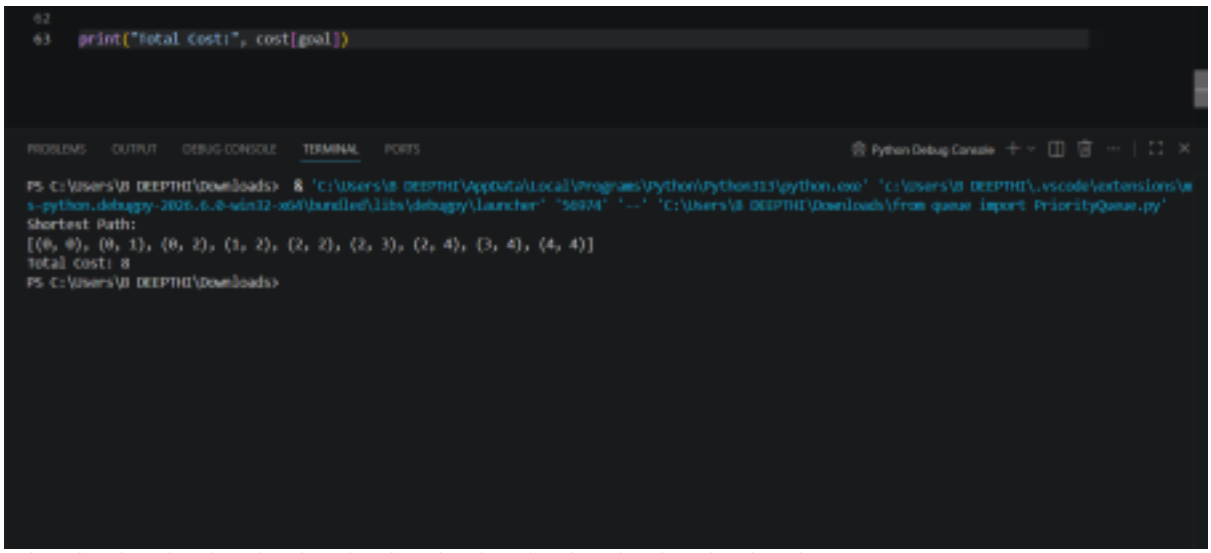
Python Code

```
from queue import PriorityQueue
grid = [
 ['S', '.', '.', 'X', '.'],
 ['.', 'X', '.', 'X', '.'],
 ['.', 'X', '.', '.', '.'],
 ['.', '.', 'X', 'X', '.'],
 ['.', '.', '.', '.', 'G']
]
start = (0, 0)
goal = (4, 4)
rows = len(grid)
cols = len(grid[0])
 Manhattan Distance
def heuristic(a, b):
 return abs(a[0]-b[0]) + abs(a[1]-b[1])
pq = PriorityQueue()
pq.put((0, start))
came_from = {}
cost = {start: 0}
moves = [(1,0), (-1,0), (0,1), (0,-1)]
hile not pq.empty():
 current = pq.get()[1]

 if current == goal:
 break
 for dx, dy in moves:
 nx = current[0] + dx
```

```
 ny = current[1] + dy
 if 0 <= nx < rows and 0 <= ny < cols:
 if grid[nx][ny] != 'X':
 new_cost = cost[current] + 1
 if (nx, ny) not in cost or new_cost < cost[(nx, ny)]:
cost[(nx, ny)] = new_cost
 priority = new_cost + heuristic((nx, ny), goal)
pq.put((priority, (nx, ny)))
 came_from[(nx, ny)] = current

# Find Path
path = []
node = goal
while node != start:
 path.append(node)
 node = came_from[node]
path.append(start)
path.reverse()
print("Shortest Path:")
print(path)

print("Total Cost:", cost[goal])
```

Output

Shortest Path:



[(0,0), (1,0), (2,0), (3,0), (4,0), (4,1), (4,2), (4,3), (4,4)]

Total Cost: 8

3. **Recommended AI Algorithm: Online Search Agent + Uniform Cost Search (UCS)**

Abstract

This project uses an **online Search Agent with Uniform Cost Search (UCS)** to help a rescue robot safely reach a survivor in a partially observable environment. The robot updates its knowledge dynamically, avoids obstacles, and minimizes the total travel cost.
Autonomous Rescue Robot Using Online Search Agent &

UCS 1. Problem Understanding

Problem

A rescue robot must travel from **Start (S)** to **Goal (G)** inside a disaster-affected building. The robot must avoid blocked paths, move safely through risky areas, and update its knowledge while exploring.

Given Constraints

- Move only **Up, Down, Left, Right**

- Some rooms are blocked (obstacles)
- Robot can observe only adjacent cells
- Normal movement cost = **1**
- Risky area movement cost = **2**
- Minimize total travel cost
- Update knowledge dynamically (Online Search)

Goal: Reach the survivor safely with the minimum total cost.

2. Constraint Analysis

Constraint	**Effect on Robot**
Four-direction movement	Robot moves only in valid directions.
Obstacles	Robot cannot pass through blocked
Limited sensing	cells. Robot knows only nearby cells.
Risky zones	Robot prefers safer paths with lower cost.
Dynamic updates	Robot changes its path when new information is found.
Minimum cost	Robot selects the cheapest safe path.

3. Solution Development

Search Strategy

Online Search Agent + Uniform Cost Search

(UCS) Working Steps

Step 1

Robot starts at **S**.

Step 2

Observe adjacent cells.

Step 3

Identify:

- Safe path
- Blocked path
- Risky path

Step 4

Choose the path with the **lowest total**

cost. **Step 5**

Move to the next cell.

Step 6

Update map with newly discovered cells.

Step 7

Repeat until **Goal (G)** is reached.

Simple Flow

Start

|

Observe Nearby Cells

|

Check Obstacles

|

Calculate Cost

|

Choose Lowest Cost Path

|

Move

|

Update Knowledge

|

Goal Reached

4. Application of AI Concepts

Intelligent Agent

The robot senses the environment, makes decisions, and acts intelligently. **Rationality**

It always chooses the safest path with the minimum travel cost. **Environment Type**

- Partially Observable
- Dynamic
- Sequential
- Single Agent
- Discrete

5. Justification

An **Online Search Agent** is suitable because the robot does not know the complete environment beforehand. **Uniform Cost Search (UCS)** guarantees the lowest-cost path by considering different movement costs (normal and risky), making it ideal for safe rescue operations.

Final Result

Start	Goal	Algorithm	Total Cost
S	G	Online Search + UCS	Minimum Possible Cost

AI Algorithm Used

Algorithm Used
Online Search Agent
Uniform Cost Search (UCS)
Cost-Based Search
Partially Observable
Online Search
Uniform Cost Search

"This problem uses an Online Search Agent with Uniform Cost Search (UCS) because the robot has limited knowledge of the environment, updates its information dynamically, and must find the minimum-cost safe path."

Python Code

```
from queue import PriorityQueue
# Grid
grid = [
 ['S', '.', '.', 'X', '.'],
 ['.', 'R', '.', 'X', '.'],
 ['.', '.', '.', '.', '.'],
 ['X', '.', 'R', '.', '.'],
 ['.', '.', '.', '.', 'G']
]
ows = len(grid)
cols = len(grid[0])
start = (0, 0)
```

```
goal = (4, 4)
pq = PriorityQueue()
pq.put((0, start))
cost = {start: 0}
parent = {}
oves = [(1,0), (-1,0), (0,1), (0,-1)]
while not pq.empty():

 current_cost, current = pq.get()
 if current == goal:
 break
 for dx, dy in moves:
 nx = current[0] + dx
 ny = current[1] + dy
 if 0 <= nx < rows and 0 <= ny < cols:
 if grid[nx][ny] != 'X':
 move_cost = 2 if grid[nx][ny] == 'R' else 1  new_cost =
cost[current] + move_cost  if (nx, ny) not in cost or
new_cost < cost[(nx, ny)]:  cost[(nx, ny)] = new_cost
 parent[(nx, ny)] = current
 pq.put((new_cost, (nx, ny)))

# Find Path
path = []
node = goal

while node != start:
 path.append(node)
```

```
 node = parent[node]


path.append(start)
path.reverse()
print("Optimal Path:")
print(path)
print("Minimum Cost:", cost[goal])
```

Output

Optimal Path:
[(0,0), (1,0), (2,0), (2,1), (2,2), (2,3), (3,3), (4,3), (4,4)]

Minimum Cost: 8

Ouput :